

EXPERIMENT 4:INTENSITY TRANSFORMATION AND HISTOGRAM EQUALIZATION

AIM:

To perform intensity transformation and histogram equalization using python in google colab.

MATERIALS REQUIRED:

Google colab software(python)

CODE:

INTENSITY TRANSFORMATION

1)Image negatives:

Image negatives are discussed in this [article](#). Mathematically, assume that an image goes from intensity levels 0 to (L-1). Generally, $L = 256$. Then, the negative transformation can be described by the expression $s = L-1-r$ where r is the initial intensity level and s is the final intensity level of a pixel. This produces a photographic negative.

```
import cv2

import matplotlib.pyplot as plt

# Read an image

img_bgr = cv2.imread('human bone fracture.jpg', 1)

plt.imshow(img_bgr)

plt.show()

# Histogram plotting of the image

color = ('b', 'g', 'r')

for i, col in enumerate(color):

    histr = cv2.calcHist([img_bgr],

                        [i], None,

                        [256],

                        [0, 256])
```

```

plt.plot(histr, color = col)
# Limit X - axis to 256
plt.xlim([0, 256])
plt.show()

# get height and width of the image
height, width, _ = img_bgr.shape
for i in range(0, height - 1):
    for j in range(0, width - 1):
        # Get the pixel value
        pixel = img_bgr[i, j]
        # Negate each channel by
        # subtracting it from 255
        # 1st index contains red pixel
        pixel[0] = 255 - pixel[0]
        # 2nd index contains green pixel
        pixel[1] = 255 - pixel[1]
        # 3rd index contains blue pixel
        pixel[2] = 255 - pixel[2]
        # Store new values in the pixel
        img_bgr[i, j] = pixel

# Display the negative transformed image
plt.imshow(img_bgr)
plt.show()

# Histogram plotting of the
# negative transformed image
color = ('b', 'g', 'r')
for i, col in enumerate(color):

```

```

histr = cv2.calcHist([img_bgr],
                     [i], None,
                     [256],
                     [0, 256])

```

```

plt.plot(histr, color = col)
plt.xlim([0, 256])
plt.show()

```

OUTPUT:



2) Log Transformations -

Mathematically, log transformations can be expressed as $s = c \log(1+r)$. Here, s is the output intensity, $r \geq 0$ is the input intensity of the pixel, and c is a scaling constant. c is given by $255/(\log(1 + m))$, where m is the maximum pixel value in the image. It is done to ensure that the final pixel value does not exceed $(L-1)$, or 255. Practically, log transformation maps a narrow range of low-intensity input values to a wide range of output values. Consider the following input image.

```

import cv2

import numpy as np

# Open the image.

img = cv2.imread('human bone fracture.jpg')

# Apply log transform.

c = 255/(np.log(1 + np.max(img)))

```

```
log_transformed = c * np.log(1 + img)
# Specify the data type.
log_transformed = np.array(log_transformed, dtype = np.uint8)
# Save the output.
cv2.imwrite('log_transformed.jpg', log_transformed)
```

OUTPUT:



3) Power-Law (Gamma) Transformation -

Power-law (gamma) transformations can be mathematically expressed as $s = cr^\gamma$. Gamma correction is important for displaying images on a screen correctly, to prevent bleaching or darkening of images when viewed from different types of monitors with different display settings. This is done because our eyes perceive images in a gamma-shaped curve, whereas cameras capture images in a linear fashion. Below is the Python code to apply gamma correction.

```
import cv2
import numpy as np
# Open the image.
img = cv2.imread('human bone fracture.jpg')
# Trying 4 gamma values.
for gamma in [0.1, 0.5, 1.2, 2.2]:
```

```
# Apply gamma correction.
```

```
gamma_corrected = np.array(255*(img / 255) ** gamma, dtype = 'uint8')
```

```
# Save edited images.
```

```
cv2.imwrite('gamma_transformed'+str(gamma)+'.jpg', gamma_corrected)
```

OUTPUT:

Gamma = 0.1:



Gamma = 0.5:



Gamma = 1.2:



Gamma = 2.2:



4) Piecewise-Linear Transformation Functions -

These functions, as the name suggests, are not entirely linear in nature. However, they are linear between certain x-intervals. One of the most commonly used piecewise-linear transformation functions is contrast stretching. Contrast can be defined as:

$$\text{Contrast} = (I_{\text{max}} - I_{\text{min}}) / (I_{\text{max}} + I_{\text{min}})$$

With $(r1, s1)$, $(r2, s2)$ as parameters, the function stretches the intensity levels by essentially decreasing the intensity of the dark pixels and increasing the intensity of the light pixels. If $r1 = s1 = 0$ and $r2 = s2 = L-1$, the function becomes a straight dotted line in the graph (which gives no effect). The function is monotonically increasing so that the order of intensity levels between pixels is preserved. Below is the Python code to perform contrast stretching.

```
import cv2
```

```
import numpy as np
```

```
# Function to map each intensity level to output intensity level.
```

```
def pixelVal(pix, r1, s1, r2, s2):
```

```
    if (0 <= pix and pix <= r1):
```

```
        return (s1 / r1)*pix
```

```
    elif (r1 < pix and pix <= r2):
```

```
        return ((s2 - s1)/(r2 - r1)) * (pix - r1) + s1
```

```
    else:
```

```
        return ((255 - s2)/(255 - r2)) * (pix - r2) + s2
```

```
# Open the image.
img = cv2.imread('human bone fracture.jpg')

# Define parameters.
r1 = 70
s1 = 0
r2 = 140
s2 = 255

# Vectorize the function to apply it to each value in the Numpy array.
pixelVal_vec = np.vectorize(pixelVal)

# Apply contrast stretching.
contrast_stretched = pixelVal_vec(img, r1, s1, r2, s2)

# Save edited image.
cv2.imwrite('contrast_stretch.jpg', contrast_stretched)
```

OUTPUT:



HISTOGRAM EQUALIZATION

- Importing Libraries

Here we will be importing [OpenCV](#), [Numpy](#) and [Matplotlib](#) libraries for the implementation.

```
import cv2
```

```
import numpy as np
```

```
import matplotlib.pyplot as plt
```

- Applying Histogram Equalization

Now first we load the image, for histogram equalization we typically work with grayscale images and then apply histogram equalization to improve its contrast and stack the original and equalized images for comparison. Here we will be using a random sample image which you can download from [here](#), also you can use any image.

- **img = cv2.imread('human bone fracture.jpg', 0):** It reads the image from the specified path and the second argument 0 ensures the image is read in grayscale mode which is required for histogram equalization.
- **np.hstack():** It horizontally stacks the original image (img) and the equalized image (equ) for easy comparison.

```
img = cv2.imread('human bone fracture.jpg', 0)
```

```
equ = cv2.equalizeHist(img)
```

```
res = np.hstack((img, equ))
```

- Displaying the Result

Finally, we display the stacked images using matplotlib.

```
plt.figure(figsize=(10, 5))
```

```
plt.imshow(res, cmap='gray')
```

```
plt.title("Original vs Equalized Image")
```

```
plt.axis('off')
```

```
plt.show()
```

Output:

